

MiniRISC Assignment 1A

ISA Planning and Processor Architecture

Computer Organization and Architecture Laboratory
Department of Computer Science and Engineering, IIT Kharagpur

Anshumaan Mishra (24CS10082)
Saksham Kumar (24CS10029)

Submitted for the deadline of 5 October 2026

1 Processor architecture

Figure 1 is the block diagram of the planned 32-bit MiniRISC processor. The datapath is drawn with solid boxes and solid lines. The control path is the control unit at the top together with the dashed lines coming out of it. The control signals that each block receives are written inside that block in typewriter font. The multiplier is only a placeholder attached to the ALU.

1.1 Description of the blocks

Program counter and next-address logic. The PC is 32 bits wide and counts words, so sequential execution is $PC \leftarrow PC + 1$. The next-address logic checks the branch condition from the ALU flags and uses one adder to compute $PC + 1$ or $PC + 1 + \text{sxt}(\text{offset})$. The same adder output is the link address saved by JAL. The final multiplexer is controlled by `pc_src` and picks the adder output (00), the jump target `jta` (01), the register `rs` for JR (10), or zero for start-up (11).

Instruction memory. A single-port BRAM ROM, one 32-bit instruction per word address. It is addressed with `next_pc`, so the PC and the fetched instruction change on the same clock edge. This hides the one-cycle read latency of the BRAM. `ic_enable` holds the output while a multi-cycle instruction is running.

Register file. 16 registers of 32 bits, two read ports and one write port. R0 always reads zero and writes to it are ignored. The read address of the `rs` port passes through a 2:1 multiplexer (`force_rs_zero`) and the read address of the `rt` port passes through a multiplexer controlled by `rt_sel`. The reason for these two multiplexers is given in Section 3.

ALU and input multiplexers. The x input is always the `rs` read port. The y input comes from the ALU y multiplexer, which selects either the `rt` read port or the extended immediate (`alu_src`). The ALU has load-upper, compare, arithmetic, logic and shift units. `alu_func[5:3]` selects the unit and `alu_func[2:0]` selects the operation inside it.

Data memory. A single-port BRAM RAM. The address is the ALU result and the store data is the `rt` read port. The read is synchronous, so LD needs two cycles.

Write-back. A multiplexer (`reg_in_src`) selects the ALU result, the memory data or $PC + 1$ and feeds the single write port of the register file.

Control unit. A four-state FSM (IDLE, RUN, WAIT, DONE). Its inputs are the instruction `opcode` and `fn` fields, `start` and `reset`. In IDLE the PC is forced to 0 so that instruction 0 is already fetched when `start` arrives. Most instructions finish in RUN. LD (and later the multiplier) goes to WAIT, where the PC and the instruction are held until the result is ready. HALT and illegal opcodes go to DONE.

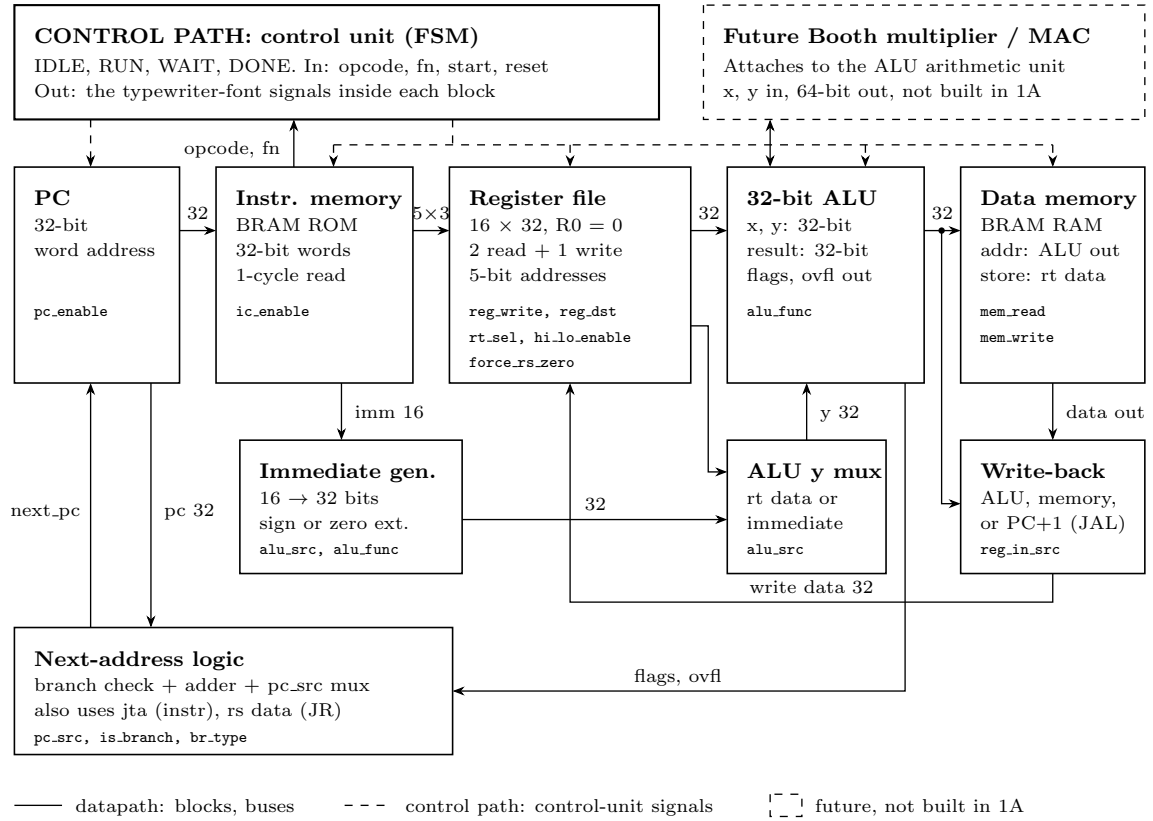


Figure 1: Block diagram of MiniRISC. Solid boxes and lines: datapath. Control unit and dashed lines: control path. Dashed box at the top right: placeholder for the Booth multiplier / MAC.

1.2 Required diagram elements

No.	Required element	Where it is in Figure 1
1	PC and PC-update path	PC, next-address logic, <code>pc</code> and <code>next_pc</code> buses
2	Instruction input / instruction-memory interface	Instr. memory, 32-bit instruction, <code>ic_enable</code>
3	Instruction decoder / control unit	Control unit, fed by <code>opcode</code> , <code>fn</code>
4	16×32 register file, 2 read ports, 1 write port	Register file
5	Immediate-generation logic	Immediate gen.
6	ALU input multiplexers	ALU y mux; <code>force_rs.zero</code> and <code>rt_sel</code> address multiplexers
7	32-bit ALU	32-bit ALU
8	Branch-decision / PC-selection path	Next-address logic, ALU flags, <code>pc_src</code> , <code>is_branch</code> , <code>br_type</code>
9	Data-memory interface	Data memory, <code>mem.read</code> , <code>mem.write</code>
10	Write-back path	Write-back multiplexer and write-data bus to the register file
11	Future multiplier / MAC integration point	Dashed block above the ALU
12	Major buses and widths	Labels on the buses
13	Major control signals	Typewriter-font names in each block

1.3 Basic processor requirements

Requirement	How it is met
Sixteen 32-bit registers R0–R15	16×32 array. R15 is also used as the link register by JAL
Two read ports, one write port	Yes. The write port is fed by the write-back multiplexer
R0 hard-wired to zero	Reads as 0, writes are ignored
32-bit PC	Yes, counts words
32-bit ALU	Load-upper, compare/set, arithmetic, logic and shift units
Decode and control logic	Fields are taken directly from the instruction word; control-unit FSM
Immediate generation	16 to 32 bits, sign or zero extension
Load/store, branch and program control	LD, ST; compare-and-branch, J, JAL, JR, NOP, HALT
Provision for the Booth multiplier	MUL/MULU slot in the ALU arithmetic unit, HI/LO registers, hardware stall
Instruction and data memory interfaces	Single-port BRAM ROM and BRAM RAM

HI and LO are two extra 32-bit registers kept next to the 16×32 array. They are written only by MULU and read only by MFHI and MFLO. The general purpose register file seen by the programmer is therefore exactly 16×32 with R0 = 0.

2 Complete planned instruction list

Group	Mnemonic	Syntax	Operation
Control	NOP	NOP	$PC \leftarrow PC + 1$
Control	HALT	HALT	stop, PC frozen until reset
Arithmetic	ADD, SUB	ADD rd, rs, rt	$rd \leftarrow rs + rt, rd \leftarrow rs - rt$
Arithmetic	ADDI, SUBI	ADDI rd, rs, imm	$rd \leftarrow rs \pm \text{sext}(imm)$
Arithmetic	MUL	MUL rd, rs, rt	$rd \leftarrow \text{low 32 bits of } rs \times rt \text{ (signed)}$
Arithmetic	MULU	MULU rs, rt	$\{HI, LO\} \leftarrow rs \times rt \text{ (64 bit, unsigned)}$
Logic	AND, OR, NOR, XOR	AND rd, rs, rt	bitwise operation on rs and rt
Logic	NOT	NOT rd, rs	$rd \leftarrow \sim rs$
Logic immediate	ANDI, ORI, NORI, XORI	ORI rd, rs, imm	$rd \leftarrow rs \text{ op } \text{zext}(imm)$
Shift	SLL, SRL, SRA	SLL rd, rs, rt	shift rs left, logical right, arithmetic right by rt
Shift immediate	SLLI, SRLI, SRAI	SLLI rd, rs, imm	shift rs by imm
Set on compare	SLT, SGT, SLE, SGE, SEQ, SNE	SLT rd, rs, rt	$rd \leftarrow 1$ if the relation holds, else 0
Set immediate	SLTI, SGTI, SLEI, SGEI, SEQI, SNEI	SLTI rd, rs, imm	same, against $\text{sext}(imm)$
Constants	LI	LI rd, imm	$rd \leftarrow \text{sext}(imm)$
Constants	LUI	LUI rd, imm	$rd \leftarrow imm \ll 16$
Transfer	MOVE	MOVE rd, rs	$rd \leftarrow rs$

Group	Mnemonic	Syntax	Operation
HI/LO read	MFHI, MFLO	MFHI <i>rd</i>	$rd \leftarrow HI, rd \leftarrow LO$
Memory	LD	LD <i>rd</i> , imm(<i>rs</i>)	$rd \leftarrow MEM[rs + sext(imm)]$
Memory	ST	ST <i>rd</i> , imm(<i>rs</i>)	$MEM[rs + sext(imm)] \leftarrow rd$
Jump	J	J <i>target</i>	$PC \leftarrow target$ (26-bit word address)
Jump	JAL	JAL <i>target</i>	$R15 \leftarrow PC + 1; PC \leftarrow target$
Jump	JR	JR <i>rs</i>	$PC \leftarrow rs$
Branch	BEQ, BNE	BEQ <i>rs</i> , <i>rd</i> , <i>label</i>	if $rs = rd$ ($\neq$): $PC \leftarrow PC + 1 + sext(offset)$
Branch	BLT, BLE, BGT, BGE	BLT <i>rs</i> , <i>rd</i> , <i>label</i>	same, when $rs < rd, \leq, >, \geq$
Branch	BZ	BZ <i>rs</i> , <i>label</i>	branch if $rs = 0$
Branch	BV	BV <i>rs</i> , <i>rd</i> , <i>label</i>	branch if $rs - rd$ overflows

There are 53 instructions in total, counting each opcode or function code once.

2.1 Coverage of the required classes

Required class	MiniRISC instructions
Arithmetic	ADD, SUB, ADDI, SUBI. INC and DEC are ADDI <i>rd</i> , <i>rd</i> , 1 and SUBI <i>rd</i> , <i>rd</i> , 1
Logical	AND, OR, XOR, NOR, NOT, and ANDI, ORI, XORI, NORI
Shift, amount from register or immediate	SLL, SRL, SRA (register amount); SLLI, SRLI, SRAI (immediate amount)
Comparison	SLT, SGT, and also SLE, SGE, SEQ, SNE with immediate forms
Memory	LD, ST with base register plus offset
Branch and control	J, JR; BZ and six compare-and-branch forms; NOP; HALT. BMI is BLT <i>rs</i> , <i>R0</i> , <i>label</i> and BPL is BGE <i>rs</i> , <i>R0</i> , <i>label</i>
Register transfer, constants	MOVE, LI, LUI. LUI followed by ORI builds any 32-bit constant
Multiplication	MUL, MULU, with MFHI and MFLO to read the 64-bit result

2.2 Additional instructions and why they were added

Instruction(s)	Reason
SLE, SGE, SEQ, SNE and immediate forms	All six relations come from the same compare unit, so loop conditions need no extra inversion
BEQ, BNE, BLT, BLE, BGT, BGE	Compare two registers and branch in one instruction, so no flag register is needed. BZ, BMI and BPL are special cases
BV	Branches on signed overflow of $rs - rd$. Useful for checking results in the later multiply and accumulate programs
JAL, JR	Subroutine call and return for the matrix and image programs
MULU, MFHI, MFLO	Make the whole 64-bit product visible and give a place to accumulate later

3 Instruction formats

All instructions are 32 bits. Every field is at the same position in every format. The destination register is always in bits 25:21.

	31:26	25:21	20:16	15:11	10:6	5:0
R	op	rd	rs	rt	shamt (reserved)	fn
I	op	rd	rs	imm[15:0]		
Branch	op	rs2	rs	offset[15:0]		
J	op	jta[25:0]				

The I format is used by the ALU immediate instructions, LI, LUI, LD and ST. The 16-bit immediate occupies the same bits as the **rt**, **shamt** and **fn** fields of the R format.

Source and destination fields. The destination is **rd** (25:21) and the sources are **rs** (20:16) and **rt** (15:11). In the I format the second operand is the immediate. For **ST** the **rd** field names the register whose value is stored. In the branch format the first register field (**rs2**) is the second register of the comparison and nothing is written.

Function field. Only R-type instructions (op = 100000) use bits 5:0 as the ALU function. In every other format these bits are part of the immediate, so each I-type instruction has its own opcode.

Shift fields. The shift amount is taken from the second ALU operand, which is the register **rt** for R-type and the immediate for I-type. Only the low five bits are used. The **shamt** field is reserved and not used.

Control instructions. NOP has opcode 000000. HALT has opcode 111111.

Why the register read addresses have multiplexers. The immediate and branch formats use bits 15:0 for the constant, so the **rt** field does not exist there. Some of these instructions still need a second register. **ST** needs the value to be stored, and the compare-and-branch instructions need the second register to compare. In both cases the **rd** field is free (nothing is written), so the **rt** read address is taken from **rd** (**rt_sel** = 001). **MOVE** and **BZ** need a constant zero as the second operand (**rt_sel** = 010 selects R0). **MFHI** and **MFLO** read HI and LO through the same port (011 and 100). Using a multiplexer on the existing port avoids a third read port.

On the **rs** side, **force_rs_zero** replaces the x operand by R0. A constant can only enter the ALU as y, so **LI** is computed as $0 + imm$, and **MFHI** and **MFLO** as $0 + HI$ and $0 + LO$. All of these reuse the ordinary add operation.

3.1 Encoding of every instruction

The table lists, for each instruction, the format, the 6-bit opcode, the **fn** value (R-type ALU instructions only) and the fields that carry operands. In the assembly syntax of the branches (**BEQ rs, rd, label**) the first register is placed in the **rs** field (20:16) and the second register is placed in the **rs2** field (25:21). Fields that are not listed are written as zero by the assembler and ignored by the hardware. **MOVE**, **MFHI**, **MFLO** and **JR** use the R layout but do not look at **fn**.

Mnemonic	Format	op	fn	Fields used
ADD	R	100000	010000	rd, rs, rt
SUB	R	100000	010001	rd, rs, rt
MUL	R	100000	010010	rd, rs, rt
MULU	R	100000	010011	rs, rt (rd = 0)
AND	R	100000	011000	rd, rs, rt
OR	R	100000	011001	rd, rs, rt
NOT	R	100000	011010	rd, rs
NOR	R	100000	011011	rd, rs, rt
XOR	R	100000	011100	rd, rs, rt

Mnemonic	Format	op	fn	Fields used
SLL	R	100000	100000	rd, rs, rt
SRL	R	100000	100001	rd, rs, rt
SRA	R	100000	100010	rd, rs, rt
SLT	R	100000	001000	rd, rs, rt
SGT	R	100000	001001	rd, rs, rt
SLE	R	100000	001010	rd, rs, rt
SGE	R	100000	001011	rd, rs, rt
SEQ	R	100000	001100	rd, rs, rt
SNE	R	100000	001101	rd, rs, rt
MOVE	R	100100	–	rd, rs
MFHI	R	100001	–	rd
MFLO	R	100111	–	rd
JR	R	101011	–	rs
ADDI	I	110000	–	rd, rs, imm
SUBI	I	110001	–	rd, rs, imm
ANDI	I	110010	–	rd, rs, imm
ORI	I	110011	–	rd, rs, imm
NORI	I	110100	–	rd, rs, imm
XORI	I	110101	–	rd, rs, imm
SLLI	I	110110	–	rd, rs, imm
SRLI	I	110111	–	rd, rs, imm
SRAI	I	111000	–	rd, rs, imm
SLTI	I	111001	–	rd, rs, imm
SGTI	I	111010	–	rd, rs, imm
SLEI	I	111011	–	rd, rs, imm
SGEI	I	111100	–	rd, rs, imm
SEQI	I	111101	–	rd, rs, imm
SNEI	I	111110	–	rd, rs, imm
LI	I	100010	–	rd, imm (rs unused)
LUI	I	100011	–	rd, imm (rs unused)
LD	I	100101	–	rd, rs, imm
ST	I	100110	–	rd (value), rs (base), imm
BEQ	Branch	010000	–	rs2, rs, offset
BZ	Branch	010001	–	rs, offset (rs2 unused)
BNE	Branch	010010	–	rs2, rs, offset
BLT	Branch	010011	–	rs2, rs, offset
BLE	Branch	010100	–	rs2, rs, offset
BGT	Branch	010101	–	rs2, rs, offset
BGE	Branch	010110	–	rs2, rs, offset
BV	Branch	010111	–	rs2, rs, offset
J	J	101000	–	jta
JAL	J	101001	–	jta
NOP	–	000000	–	none
HALT	–	111111	–	none

3.2 Encoding examples

These are the machine words for the three-instruction test program of Assignment 1B and for one instruction of each remaining kind. Bits are grouped by field.

Instruction	Binary (fields)	Hex
ADDI R1, R0, 5	110000 00001 00000 0000000000000101	C0200005
ADDI R2, R0, 7	110000 00010 00000 0000000000000111	C0400007
ADD R3, R1, R2	100000 00011 00001 00010 00000 010000	80611010
MULU R2, R3	100000 00000 00010 00011 00000 010011	80021813
LUI R1, 0x1234	100011 00001 00000 0001001000110100	8C201234
LD R4, 8(R5)	100101 00100 00101 0000000000001000	94850008
ST R6, -4(R7)	100110 00110 00111 1111111111111100	98C7FFFC
BEQ R1, R2, +3	010000 00010 00001 0000000000000011	40410003
J 0x10	101000 00000000000000000000010000	A0000010
NOP	000000 00000000000000000000000000	00000000
HALT	111111 00000000000000000000000000	FC000000

4 Opcode allocation

op range	Class	Opcodes
000000	No-op	NOP
010000–010111	Conditional branches	BEQ 010000, BZ 010001, BNE 010010, BLT 010011, BLE 010100, BGT 010101, BGE 010110, BV 010111
100000–100111	R-type, constants, moves, memory	R-type 100000, MFHI 100001, LI 100010, LUI 100011, MOVE 100100, LD 100101, ST 100110, MFLO 100111
101000–101011	Jumps	J 101000, JAL 101001, JR 101011 (101010 reserved)
110000–111110	I-type ALU	ADDI 110000, SUBI 110001, ANDI 110010, ORI 110011, NORI 110100, XORI 110101, SLLI 110110, SRLI 110111, SRAI 111000, SLTI 111001, SGTI 111010, SLEI 111011, SGEI 111100, SEI 111101, SNEI 111110
111111	Stop	HALT

All unassigned opcodes are illegal and stop the processor (the FSM goes to DONE).

R-type function codes. The upper three bits of **fn** select the ALU unit and the lower three bits select the operation inside it. The I-type instructions use the same codes, produced by the control unit.

fn[5:3]	ALU unit	fn[2:0] operations
000	Load-upper	LUI 000
001	Compare/set	SLT 000, SGT 001, SLE 010, SGE 011, SEQ 100, SNE 101
010	Arithmetic	ADD 000, SUB 001, MUL 010, MULU 011
011	Logic	AND 000, OR 001, NOT 010, NOR 011, XOR 100
100	Shift	SLL 000, SRL 001, SRA 010

The first two opcode bits give the class: 11 is ALU with immediate and 01 is conditional branch. NOP is all zeros so cleared memory does nothing, and HALT is all ones. The I-type opcodes follow the order of the R-type functions. MUL, MULU and NOT have no immediate form.

5 Immediate, branch and memory-addressing conventions

Immediates. The immediate is a 16-bit field, `imm[15:0]`, extended to 32 bits. It is sign-extended for `ADDI`, `SUBI`, `LI`, the set-on-compare immediates, and for load/store and branch offsets. It is zero-extended for the logic immediates (`ANDI`, `ORI`, `NORI`, `XORI`). This is so that `LUI rd, hi16` followed by `ORI rd, rd, lo16` gives any 32-bit constant. `LUI` puts the immediate in bits 31:16 and clears the lower half. Shift immediates use the low five bits.

Branch displacement. Branch format: `op, rs2, rs, offset[15:0]`. When the branch is taken, $PC \leftarrow PC + 1 + \text{sext}(\text{offset})$. The offset counts instructions, so the range is about $\pm 32K$ instructions. The condition compares `rs` with the register in the `rs2` field, or with zero for `BZ`. There is no condition-code register and no delay slot. `J` and `JAL` use a 26-bit absolute word address and `JR` jumps to the address held in `rs`.

Memory addressing. Base register plus signed 16-bit offset. The effective address is $rs + \text{sext}(\text{imm})$ and is computed by the ALU adder. `LD rd, imm(rs)` loads one 32-bit word into `rd` and `ST rd, imm(rs)` stores `rd`. Only word accesses exist, so no alignment logic is needed.

PC convention. The instruction memory has one 32-bit instruction per address and the PC is a word address, so the PC is incremented by 1.

6 Planned MUL / MAC semantics

The ISA has `MUL` and `MULU`. A `MAC` instruction is not planned in this assignment.

	MUL	MULU
Encoding	<code>MUL Rd, Rs1, Rs2, R-type, fn = 010010</code>	<code>MULU Rs1, Rs2, R-type, fn = 010011</code>
Source registers	<code>rs, rt</code>	<code>rs, rt</code>
Destination	<code>rd</code>	HI and LO (implicit)
Signed?	Yes, two's complement operands	No, unsigned operands
Visible part of the 64-bit product	Low 32 bits, <code>product[31:0]</code>	All 64 bits. HI = <code>product[63:32]</code> , LO = <code>product[31:0]</code>
Reading the result	directly in <code>rd</code>	<code>MFHI rd, MFLO rd</code>
Multi-cycle?	Yes, expected	Yes, expected
Stall or NOPs	Hardware stall. The control FSM holds the PC and the instruction in the WAIT state until the multiplier is done (placeholder: 33 wait cycles). Software NOPs are not needed, so programs do not depend on the multiplier latency	Same as <code>MUL</code>

Integration point. The multiplier connects to the ALU arithmetic unit at the `MUL/MULU` function slot, marked in Figure 1. The upper half of the product goes to HI through the `hi_lo_enable` path. The lower half goes to LO, or to `rd` for `MUL`.

MAC later. The product is kept in HI:LO, so a `MAC` ($HI:LO \leftarrow HI:LO + rs \times rt$) can be added with a new function code in the arithmetic group and an adder on the accumulate path. No change to the instruction formats is needed.

7 Justification of the ISA organisation

- Load/store architecture with three-operand instructions and a fixed 32-bit format. Every field is taken directly from the instruction word, so decoding is simple. Only `LD` and `ST` touch memory.

- The destination is always in bits 25:21. There is one source of the write address for almost all instructions, and the 16-bit immediate fits in bits 15:0.
- The **fn** field is the ALU control code. R-type instructions need no ALU decoder, and the I-type instructions get their own opcodes (the immediate takes the place of **fn**) and reuse the same codes.
- Compare-and-branch and set-on-compare instead of a flag register. There is no hidden state between instructions, and BZ, BMI and BPL are special cases of the same hardware.
- LI, MOVE, MFHI, MFLO and BZ reuse add and subtract with an operand forced to R0. The hardware forces the zero, so these instructions do not depend on the assembler leaving the unused fields at zero.
- HI and LO keep the register file at two read ports and one write port, show the whole 64-bit product, and are ready to be used as an accumulator for a MAC.
- A word-addressed PC with $PC + 1$ needs no alignment bits. One adder gives the next sequential address, the branch target and the JAL link.
- Multi-cycle operations (LD and the multiplier) use one WAIT state in the control FSM, so the programmer never has to insert NOPs.

A Control words

Select encodings. **reg_dst**: 00 rt (not used), 01 rd, 10 LO, 11 R15. **reg_in**: 00 memory, 01 ALU, 10 PC+1, 11 zero. **pc_src**: 00 PC+1 (plus offset for a taken branch), 01 jta, 10 rs, 11 zero. **br_t**: 001 equal, 010 not equal, 011 less, 100 less or equal, 101 greater, 110 greater or equal, 111 overflow. **rt_sel**: 000 rt, 001 rd, 010 R0, 011 HI, 100 LO.

pc_en and **ic_en** are 1 for every instruction except HALT, where they are 0. For LD they are 0 in the first cycle and 1 in the second (commit) cycle. The table shows the commit cycle, except ld (**mem_read**), which is 1 only in the first cycle. X means don't care. **frz** is **force_rs_zero**. Branches use **fn** = subtract so that the ALU produces the overflow flag for BV.

Instr	op	reg_dst	hi_lo	reg_wr	alu_src	fn	ld	st	reg_in	pc_src	is_br	br_t	frz	rt_sel
NOP	000000	XX	0	0	X	XXXXXXX	0	0	XX	00	0	X	0	000
ADD	100000	01	0	1	0	010000	0	0	01	00	0	X	0	000
SUB	100000	01	0	1	0	010001	0	0	01	00	0	X	0	000
MUL	100000	01	0	1	0	010010	0	0	01	00	0	X	0	000
MULU	100000	10	1	1	0	010011	0	0	01	00	0	X	0	000
AND	100000	01	0	1	0	011000	0	0	01	00	0	X	0	000
OR	100000	01	0	1	0	011001	0	0	01	00	0	X	0	000
NOT	100000	01	0	1	0	011010	0	0	01	00	0	X	0	000
NOR	100000	01	0	1	0	011011	0	0	01	00	0	X	0	000
XOR	100000	01	0	1	0	011100	0	0	01	00	0	X	0	000
SLL	100000	01	0	1	0	100000	0	0	01	00	0	X	0	000
SRL	100000	01	0	1	0	100001	0	0	01	00	0	X	0	000
SRA	100000	01	0	1	0	100010	0	0	01	00	0	X	0	000
SLT	100000	01	0	1	0	001000	0	0	01	00	0	X	0	000
SGT	100000	01	0	1	0	001001	0	0	01	00	0	X	0	000
SLE	100000	01	0	1	0	001010	0	0	01	00	0	X	0	000
SGE	100000	01	0	1	0	001011	0	0	01	00	0	X	0	000
SEQ	100000	01	0	1	0	001100	0	0	01	00	0	X	0	000
SNE	100000	01	0	1	0	001101	0	0	01	00	0	X	0	000
ADDI	110000	01	0	1	1	010000	0	0	01	00	0	X	0	000
SUBI	110001	01	0	1	1	010001	0	0	01	00	0	X	0	000
ANDI	110010	01	0	1	1	011000	0	0	01	00	0	X	0	000
ORI	110011	01	0	1	1	011001	0	0	01	00	0	X	0	000
NORI	110100	01	0	1	1	011011	0	0	01	00	0	X	0	000
XORI	110101	01	0	1	1	011100	0	0	01	00	0	X	0	000
SLLI	110110	01	0	1	1	100000	0	0	01	00	0	X	0	000
SRLI	110111	01	0	1	1	100001	0	0	01	00	0	X	0	000
SRAI	111000	01	0	1	1	100010	0	0	01	00	0	X	0	000
SLTI	111001	01	0	1	1	001000	0	0	01	00	0	X	0	000

Instr	op	reg_dst	hi_lo	reg_wr	alu_src	fn	ld	st	reg_in	pc_src	is_br	br_t	frz	rt_sel
SGTI	111010	01	0	1	1	001001	0	0	01	00	0	X	0	000
SLEI	111011	01	0	1	1	001010	0	0	01	00	0	X	0	000
SGEI	111100	01	0	1	1	001011	0	0	01	00	0	X	0	000
SEQI	111101	01	0	1	1	001100	0	0	01	00	0	X	0	000
SNEI	111110	01	0	1	1	001101	0	0	01	00	0	X	0	000
LI	100010	01	0	1	1	010000	0	0	01	00	0	X	1	000
LUI	100011	01	0	1	1	000000	0	0	01	00	0	X	0	000
MOVE	100100	01	0	1	0	010000	0	0	01	00	0	X	0	010
MFHI	100001	01	0	1	0	010000	0	0	01	00	0	X	1	011
MFLO	100111	01	0	1	0	010000	0	0	01	00	0	X	1	100
LD	100101	01	0	1	1	010000	1	0	00	00	0	X	0	000
ST	100110	XX	0	0	1	010000	0	1	XX	00	0	X	0	001
J	101000	XX	0	0	X	XXXXXXX	0	0	XX	01	0	X	0	000
JAL	101001	11	0	1	X	XXXXXXX	0	0	10	01	0	X	0	000
JR	101011	XX	0	0	X	XXXXXXX	0	0	XX	10	0	X	0	000
BEQ	010000	XX	0	0	0	010001	0	0	XX	00	1	001	0	001
BZ	010001	XX	0	0	0	010001	0	0	XX	00	1	001	0	010
BNE	010010	XX	0	0	0	010001	0	0	XX	00	1	010	0	001
BLT	010011	XX	0	0	0	010001	0	0	XX	00	1	011	0	001
BLE	010100	XX	0	0	0	010001	0	0	XX	00	1	100	0	001
BGT	010101	XX	0	0	0	010001	0	0	XX	00	1	101	0	001
BGE	010110	XX	0	0	0	010001	0	0	XX	00	1	110	0	001
BV	010111	XX	0	0	0	010001	0	0	XX	00	1	111	0	001
HALT	111111	XX	0	0	X	XXXXXXX	0	0	XX	00	0	X	0	000